

# TopGQ: Post-Training Quantization for GNNs Leveraging Topological Similarity within Nodes

Anonymous Authors<sup>1</sup>

## Abstract

Large-scale Graph Neural Networks (GNNs) suffer from high memory and computational demands, which can be alleviated by compressing full-precision values into low-bit representations. Yet, existing GNN quantization methods often require tedious retraining to mitigate performance degradation, limiting their practicality in real-world deployments. To address these challenges, we propose TopGQ, the first accurate post-training quantization framework specifically designed for GNNs. By leveraging graph topology via a novel node index (TopPIN) and introducing scale absorption to enable node-wise quantization, TopGQ eliminates the need for retraining while preserving accuracy. Extensive experiments demonstrate that TopGQ achieves QAT-level performance with an order-of-magnitude reduction in quantization overhead, establishing it as a practical and efficient solution for large-scale GNN inference.

## 1. Introduction

Graph neural networks (GNNs) attract a great amount of attention due to their ability to process diverse unstructured data. They have achieved success in numerous domains, such as recommendation systems (Pal et al., 2020; Fan et al., 2019; Zhang et al., 2023), molecular interaction (Wale et al., 2008; Borgwardt et al., 2005), transportation networks (Bai et al., 2020; Cao et al., 2020), and social network analysis (Qiu et al., 2018; Arazzi et al., 2023). However, GNNs often suffer from huge computational and memory costs due to increasing demands for processing large graphs, especially on resource-constrained devices.

One promising direction to circumvent this issue is neural

<sup>1</sup>Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

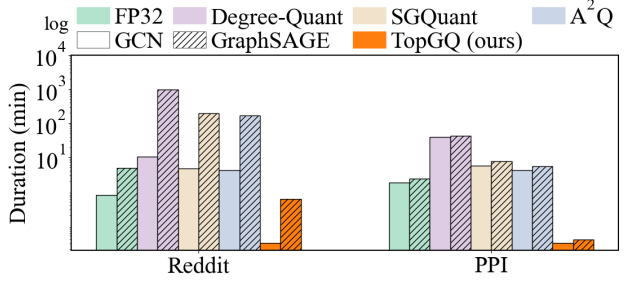


Figure 1. Comparing duration of existing GNN quantization methods against TopGQ.

network quantization (Choukroun et al., 2019; Zhao et al., 2019; Choi et al., 2021), which efficiently reduces the computation and memory requirements of GNN inference by utilizing reduced numerical precision. Despite its advantages, applying quantization to GNNs presents unique challenges due to the highly diverse magnitudes of vertex features. This is known to be caused by the necessary message-passing of the GNN algorithm, which aggregates features from neighboring vertices. Since the quantization process is known to be highly sensitive to the magnitude outliers (Wei et al., 2022), such diversity in aggregated features in GNNs leads to substantial quantization errors.

To handle the problem of diverse magnitudes, several methods (Tailor et al., 2020; Zhu et al., 2022) adopt quantization-aware training (QAT). They reduce quantization error by considering the degree information (Tailor et al., 2020) or applying mixed-precision strategy (Zhu et al., 2022). However, QAT methods accompany huge computation and memory costs for the quantization process, requiring excessive resources and time larger than full-precision GNN pretraining.

As shown in Figure 1 prior GNN quantization methods (Tailor et al., 2020; Zhu et al., 2022) consume significant amount of time in QAT. In the case of Reddit dataset for GraphSAGE, it could take hours whereas the fp32 pertaining took only a few minutes. This poses a major scalability issue, especially as the graph sizes grow.

This inefficiency hinders the deployment of GNN quantiza-

tion in real-world applications, particularly for continuously evolving graphs such as traffic networks (Jiang & Luo, 2022; Sharma et al., 2023) and social media graphs (Gao et al., 2023; Yao et al., 2023). Additionally, safety-critical applications, such as fraud detection in financial transactions (Lu et al., 2022; Zhou et al., 2021; Liu et al., 2018), require rapid updates, further necessitating efficient and fast quantization techniques.

To tackle this issue, we propose TopGQ, the first accurate post-training quantization (PTQ) framework for GNNs. Unlike previous QAT-based approaches, TopGQ eliminates the need for gradient computation or parameter updates, significantly reducing the computational and memory overhead associated with quantization.

The primary challenge of PTQ lies in preserving near-original performance without any retraining. To achieve this goal, we opt to use node-wise quantization during the aggregation phase of GNN inference. We observe that the value range often vary more across nodes than across hidden features. Thus, especially for PTQ where retraining is infeasible, node-wise quantization is crucial.

While the idea of node-wise quantization is not new, two challenges arise in using this for aggregation to attain the throughput advantage of quantization. First, the node dimension corresponds to the inner dimension of the matrix multiplication in aggregation, preventing the use of integer arithmetic hardware units. Second, in the inductive settings, there must be a systematic method to assign quantization parameters (i.e., scale and zero-point) to unseen nodes.

TopGQ proposes two key techniques to address the aforementioned issues: (1) *selective scale absorption* technique, which merges node-specific scaling factors into the adjacency matrix to allow fast low-precision matrix multiplication, and (2) TopPIN, a node-dependent index that encodes local topological information that allows us to efficiently assign quantization parameters to previously unseen nodes at a low overhead.

Our extensive experiments show that TopGQ outperforms current state-of-the-art methods for GNN quantization, achieving up to 1666 $\times$  faster quantization time without sacrificing accuracy or inference latency. These results establish a new performance standard in GNN quantization.

## Contributions.

- We present TopGQ, the first PTQ framework for GNNs, surpassing existing training-based methods in accuracy and achieving up to 1666 $\times$  speedup in the quantization process.
- We propose scale absorption, a technique that merges node-wise scaling factors into adjacency matrices, en-

abling efficient low-precision arithmetic for node-wise quantization.

- We introduce TopPIN, a theoretically grounded node index that captures local graph topology, ensuring nodes with similar topological properties share similar quantization parameters.

## 2. Background

### 2.1. Graph Neural Networks

Let  $G = (V, E)$  be a directed graph with  $n = |V|$  vertices labeled  $v_1, \dots, v_n$ . Denote by  $A \in \mathbb{R}^{n \times n}$  the adjacency matrix, where  $A_{ij} = \mathbb{1}_{(v_j, v_i) \in E}$ . In GNNs, it is common to include self-connections by using  $A + I_n$ , where  $I_n \in \mathbb{R}^{n \times n}$  is the identity matrix. For each vertex  $v_i$ , define its closed in-neighborhood as  $\mathcal{N}(v_i) = \{v_j \mid (v_j, v_i) \in E\} \cup \{v_i\}$ , and let  $d(v_i) = |\mathcal{N}(v_i)|$ . We denote by  $D = \text{diag}(d(v_1), d(v_2), \dots, d(v_n))$  the diagonal degree matrix. Each vertex  $v_i$  is associated with a feature vector  $h_i$ .

To embed topological information into the vertex features, GNNs gather information from neighboring vertices  $v_j \in \mathcal{N}(v_i)$  to update the feature vector of  $v_i$ . This procedure is commonly referred to as the *message-passing* algorithm, which consists of two steps: *combination* and *aggregation*. First, the hidden vertex feature  $h_i^{(l)}$  is multiplied by the weight matrix  $W^{(l)}$  in the  $l$ -th GNN layer (combination). Next, the feature is updated (aggregation) to  $h_i^{(l+1)}$  as follows:

$$h_i^{(l+1)} = \phi\left(\bigoplus_{\{j \mid v_j \in \mathcal{N}(v_i)\}} W^{(l)} h_j^{(l)}\right), \quad (1)$$

where  $\phi$  is a feature update function, and  $\bigoplus$  denotes a permutation-invariant operator such as *sum* or *mean*.

The GNN computation can also be formulated in matrix form. Let  $X^{(l)} \in \mathbb{R}^{n \times d_l}$  be the matrix of vertex features at layer  $l$ , where  $X^{(l)} = [h_1^{(l)}, \dots, h_n^{(l)}]^T$ , and let  $W^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$  be the corresponding weight matrix. Then, using an augmented adjacency matrix  $\tilde{A} \in \mathbb{R}^{n \times n}$ , the combination and aggregation steps can be written as:

$$X_c^{(l)} = W^{(l)} X^{(l)}, \quad X^{(l+1)} = \sigma(\tilde{A} X_c^{(l)}), \quad (2)$$

where  $\sigma$  is a nonlinear activation function. The specific form of  $\tilde{A}$  varies by GNN architecture. For example, GCN (Kipf & Welling, 2016) employs the normalized graph Laplacian  $\tilde{A} = D^{-1/2}(A + I_n)D^{-1/2}$ , while GIN (Xu et al., 2019) uses the binary matrix  $\tilde{A} = A + I_n$ . GraphSAGE (Hamilton et al., 2017) differs by sampling a subset of neighbors instead of using the entire neighborhood during aggregation.

**Transductive and Inductive Settings.** GNN settings can be generally divided into two settings: transductive and inductive. In the transductive setting, the graphs used for both

training and inference are identical. All the test nodes are available at training time, except their labels (e.g., classes). As a result, the inference time of a transductive GNN is typically very short. In the case of node classification, it involves only an MLP classifier on the already computed embedding of the test node. Because of this, inference systems often involve simple lookups on the stored results (Xu et al., 2024), giving quantization little practical value.

In contrast, the inductive setting provides unseen nodes or even entirely new graphs at test time. This requires computing all intermediate embeddings through a full forward pass on the graph. Consequently, GNN quantization would have a much greater impact in inductive settings.

## 2.2. Quantization

Quantization replaces high-precision floating-point operations with low-bit integer operations, thereby reducing computational complexity and memory usage. In this work, we adopt simple yet effective uniform integer quantization with scale ( $s$ ) and zero-point ( $z$ ). Given a matrix  $X$ , each element  $x \in X$  is quantized as:

$$x^q = Q(x; s, z) = \text{clamp}\left(\left\lfloor \frac{1}{s} \cdot (x - z) \right\rfloor, q_{\min}, q_{\max}\right), \quad (3)$$

$$s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}, \quad (4)$$

where  $k$  is the quantization bitwidth,  $q_{\min}$  and  $q_{\max}$  are the minimum and maximum integer values in the  $k$ -bit representation, and  $\lfloor \cdot \rfloor$  denotes rounding. In *symmetric* quantization, the representation range is centered at zero (i.e.,  $z = 0$ ). By contrast, *asymmetric* quantization shifts zero-point to  $z = x_{\min} - s \cdot q_{\min}$ . Furthermore, row-wise or column-wise quantization is often used to handle outliers more effectively. In these schemes, each row (or column) has its own quantization parameters ( $s, z$ ), computed independently.

Two mainstream approaches have been extensively studied for quantization: post-training quantization (PTQ) and quantization-aware training (QAT). PTQ goes through a calibration process which adjusts the scale, zero-point using only a small set of data. Conversely, QAT methods directly apply gradient-based training to explicitly reduce the quantized network’s target loss. The major discrepancy between them is that QAT generally incorporates updating weight parameters, while PTQ methods focus on quantization parameters without weight updates and is much faster.

## 2.3. Quantization of Graph Neural Networks

In the context of graph neural networks (GNNs), efficient inference must consider both the *combination* phase and the

*aggregation* phase.

$$X_c^{(l)} = Q(W; S_W, Z_W) \cdot Q(X^{(l)}; S_{X^{(l)}}, Z_{X^{(l)}}), \quad (5)$$

$$X^{(l+1)} = \sigma\left(Q(\tilde{A}; S_{\tilde{A}}, Z_{\tilde{A}}) \cdot Q(X_c^{(l)}; S_{X_c^{(l)}}, Z_{X_c^{(l)}})\right). \quad (6)$$

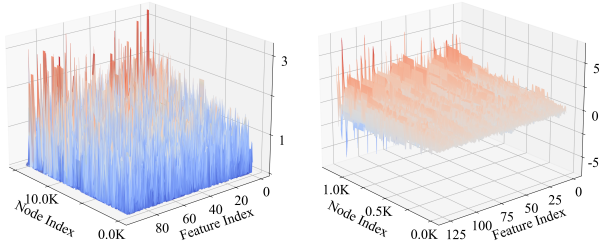
Here,  $W$ ,  $X^{(l)}$ ,  $X_c^{(l)}$ , and  $\tilde{A}$  may each follow different quantization policies (e.g., per-tensor, row-wise, or column-wise), and these design decisions greatly affect the final accuracy and inference efficiency.

For example, Degree-Quant (Tailor et al., 2020) employs per-tensor quantization for all matrices in both the combination and aggregation phases. Although this choice uses minimal storage for quantization parameters, it can be vulnerable to outliers that significantly affect the global scale. In contrast,  $A^2Q$  (Zhu et al., 2022) mitigates outlier effects by assigning column-wise quantization to  $W$  and  $X_c$ , and row-wise quantization to  $X$  and  $\tilde{A}$ . As row- and column-wise quantization can separate quantization parameters of each node and feature dimension, respectively, it is more robust to outliers at the expense of increased cost.

## 3. Related Work

**GNN Quantization** can efficiently reduce extensive computational costs and memory requirements of graph neural networks (Kipf & Welling, 2016; Xu et al., 2019; Veličković et al., 2018), as they suffer from large size of real-world graphs. Degree-Quant (Tailor et al., 2020) is the first work to quantize GNN, using QAT to allow high-degree vertices to retain full-precision features at training and be quantized later for inference. EPQuant (Huang et al., 2022) focuses on reducing high memory costs incurred by processing large graph data with product quantization, along with quantized GNNs. SGQuant (Feng et al., 2020) and  $A^2Q$  (Zhu et al., 2022) are also QAT methods targeting GNN architectures, but they differ in that they allow mixed-precision to assign higher bitwidth to high-magnitude vertices. SMP (Wang et al., 2023) tackles the oversmoothing problem when quantizing deep GNNs with a customized message propagation, and (Eliasof et al., 2023) utilizes wavelet transformations in traditional image compression to quantize GNNs. Notably, existing GNN quantization methods adopt QAT, i.e., incorporating gradient-based iterative weight updates, which require significant computational overheads (Figure 1).

**Graph Topology in GNNs** is often integrated during training to help the model effectively learn the structural information (Ji, 2019; Zhang & Lu, 2020; Hu et al., 2022; Wu et al., 2018; You et al., 2021; Brasoveanu et al., 2023). For example, (Ji, 2019) uses degree centrality to find highly central vertices in their pooling layer as they are more important for effective representation learning. Also, (Zhang & Lu, 2020) uses betweenness centrality to assign weights to each



(a) GraphSAGE, IMDB-Binary (b) GCN, MUTAG

Figure 2. Magnitude plot of the feature matrix, sorted by the max value in each node for better visualization. In both settings, the distribution shows that intra-row (node) variance is lower, while the inter-row (feature) variance is higher.

node during aggregation. However, these methods do not relate topological information with node feature magnitudes, especially for quantization.

#### 4. Necessity and Challenges of Node-wise GNN Quantization

**Necessity of node-wise quantization.** Quantizing a graph neural network (GNN) requires special consideration due to its unique message-passing mechanism. In particular, the accumulation of neighborhood information induces substantial variability in per-node feature distributions, making node-wise quantization a preferred approach in complex graph topologies. As shown in Figure 2, feature magnitudes are often more effectively captured via node-wise grouping than feature-wise grouping, making node-wise quantization less prone to quantization error. The figure illustrates that outliers extend beyond individual features and are also concentrated within specific nodes.

Based on the observation, we opt to use node-wise quantization for the feature matrix  $X$  in both the combination and the aggregation phase of GNN inference. Enabling node-wise quantization in the *combination* phase is relatively straightforward. In fact, existing methods (Zhu et al., 2022; Feng et al., 2020) already employ node-wise quantization for  $X$ , leading to the following quantized matrix multiplication:

$$X \cdot W \approx \text{diag}(S_X) \cdot X^Q \cdot W^Q \cdot \text{diag}(S_W) \quad (7)$$

$$= (S_X \cdot S_W^\top) \otimes (X^Q \cdot W^Q), \quad (8)$$

where  $S_X \in \mathbb{R}^{n \times 1}$  is the node-wise scale for  $X$  and  $S_W \in \mathbb{R}^{d \times 1}$  is the feature-wise scale for  $W$ , and  $\otimes$  denotes the element-wise (Hadamard) product. Since  $X$  is quantized node-wise and  $W$  is quantized feature-wise, the quantized multiplication remains computationally efficient.

**Challenge 1: Quantization along inner dimensions.** By contrast, node-wise quantization in the *aggregation* phase is more challenging. Existing GNN quantization (Zhu et al., 2022; Feng et al., 2020) methods instead apply column-wise quantization to the intermediate activation matrix  $X_c$ , because the columns correspond to the outer dimension in message passing (see Figure 3a). While this approach is computationally convenient, it may fail to preserve the precision of activations. Specifically, applying node-wise quantization for the aggregation step,

$$\tilde{A} \cdot X_c \approx \text{diag}(S_{\tilde{A}}) \cdot \tilde{A}^Q \cdot \text{diag}(S_{X_c}) \cdot X_c^Q, \quad (9)$$

introduces the diagonal matrix  $\text{diag}(S_{X_c})$  within the multiplication, which cannot be computed using integer matrix multiplication units with common methods (Jacob et al., 2018).

To capitalize on the precision benefits of node-wise quantization while preserving computational efficiency, TopGQ proposes a novel method, scale absorption, which is detailed in Section 5.1.

**Challenge 2: Unseen nodes in inductive settings.** For quantization to have practical value on GNNs, inductive settings must be considered (Section 2.1). Contrary to transductive settings, GNN inference on inductive tasks encounters nodes unseen at training time. When node-wise quantization is used, finding the right quantization parameters for these unseen nodes is crucial for both throughput and accuracy. There can be two methods:

*On-the-Fly Computation.* A straightforward approach is to dynamically compute accurate quantization parameters for each node during inference. For each GNN layer, we scan all rows of  $X^{(l)}$  and  $X_c^{(l)}$ , then determine the minimum and maximum values per node to obtain scales and zero-points. While this ensures small quantization errors, it imposes considerable runtime overhead and considered impractical.

*Precomputed Mapping.* An alternative is to precompute a set of quantization parameters at calibration time, and map each unseen node to one of its entries, typically to one of the training set nodes. At inference, we perform a simple lookup to retrieve the appropriate parameters. Nonetheless, this requires a low-complexity *node index*  $\phi(\cdot)$  such that nodes with similar index values exhibit similar feature statistics.

TopGQ chooses a precomputed mapping approach, where we design a novel index *Topology-Aware Pairwise Index* (TopPIN) that only captures topology for lightweight computation (Section 5.2). TopPIN ensures that unseen nodes are assigned adequate quantization parameters at low inference overhead.

## 5. TopGQ Methodology

### 5.1. Selective Scale Absorption

To address the challenge of applying quantization along the inner dimensions, TopGQ introduces a novel method named scale absorption. The core idea of scale absorption is to merge node-specific scaling factors into the static adjacency matrix, effectively allowing node-wise quantization to be performed without altering the computational flow of the matrix multiplication.

The details of scale absorption is shown in Figure 3b. When we apply node-wise quantization on the aggregation Equation (9),  $\text{diag}(S_{X_c})$  matrix between the quantized matrices becomes an obstacle. To address this, we leverage the fact that  $\tilde{A}$  is a fixed topology matrix. After calculating the scale vector  $S_{X_c}$  for  $X_c$ , we proceed as follows:

$$\tilde{A} \cdot X_c \approx \tilde{A} \cdot \text{diag}(S_{X_c}) \cdot X_c^Q \quad (10)$$

$$= \tilde{A}_{X_c} \cdot X_c^Q \quad (11)$$

$$\approx \text{diag}(S_{\tilde{A}_{X_c}}) \cdot \tilde{A}_{X_c}^Q \cdot X_c^Q. \quad (12)$$

In the above, the scale diagonal matrix  $\text{diag}(S_{X_c})$  is absorbed into the adjacency matrix  $\tilde{A}$  to form  $\tilde{A}_{X_c}$ , which is then row-wisely quantized with the new scale as  $\text{diag}(S_{\tilde{A}_{X_c}}) \cdot \tilde{A}_{X_c}^Q$ . At inference time, these two terms need only be computed once and can then be applied to all layers, as both  $\tilde{A}$  and  $S_{X_c}$  depend solely on the topology of the input graph. Computation-wise, this can seem similar to SmoothQuant (Xiao et al., 2023), but they are essentially different in that scale absorption transfers scale factors to adjacent matrix to gain computational efficiency, while SmoothQuant migrates the outliers between matrices for redistribution.

TopGQ adaptively selects between node-wise and feature-wise quantization for  $X_c$ , aiming to further improve accuracy. During calibration, TopGQ evaluates both configurations by measuring the mean squared error (MSE) between the original floating-point activations and their quantized counterparts. This assessment is conducted layer by layer, and the configuration with lower MSE is stored for use during inference.

### 5.2. TopPIN: a Lightweight Index for Unseen Nodes

To support quantization in inductive settings, we devise TopPIN, a lightweight index that maps unseen nodes to existing train set nodes. We first define a node-dependent index function  $\phi$  that captures the relevant local structural properties, and then generalize it for easier computation.

**Theorem 5.1** (Node index  $\phi$  for GNN quantization). *Let  $G = (V, E)$  be an undirected graph. For  $\tilde{A}$ , we separately*

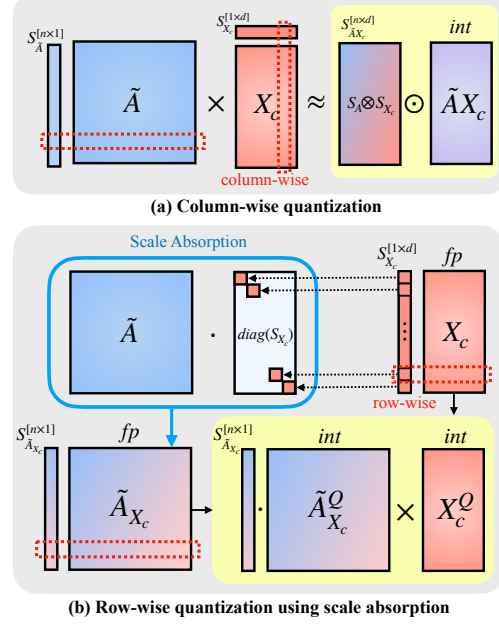


Figure 3. Comparing (a) column-wise quantization and (b) row-wise quantization using scale absorption. By absorbing the row-wise scale to the adjacency matrix, we can perform matrix multiplication in the quantized format (efficiency) whilst retaining the values in floating-point (precision).

consider unnormalized and normalized cases:

$$\tilde{A} = A + I_n \quad \text{or} \quad \tilde{A} = D^{-\frac{1}{2}}(A + I_n)D^{-\frac{1}{2}}.$$

Consider a GNN of the form

$$X^{(l+1)} = \sigma(\tilde{A} X^{(l)} W^{(l)}),$$

where  $\sigma$  is the ReLU activation, the hidden dimensions  $d_l$  are sufficiently large ( $d_l \gg 1$ ) for all layers  $l$ , and each entry of  $X^{(0)}$  and  $W^{(l)}$  is drawn independently from a distribution with zero mean and finite variance. Then, for each layer  $l$  of the GNN, define the scalar function  $\phi : V \rightarrow \mathbb{R}$  by:

**Case 1:** If  $\tilde{A} = A + I_n$  (unnormalized), then

$$\phi(v) = \sum_{v_{k_1} \in \mathcal{N}(v)} \sum_{v_{k_2} \in \mathcal{N}(v_{k_1})} \cdots \sum_{v_{k_l} \in \mathcal{N}(v_{k_{l-1}})} 1.$$

**Case 2:** If  $\tilde{A} = D^{-\frac{1}{2}}(A + I_n)D^{-\frac{1}{2}}$  (normalized), then

$$\phi(v) = \frac{1}{d(v)} \sum_{k_1 \in \mathcal{N}(v)} \left[ \frac{1}{d(v_{k_1})^2} \sum_{k_2 \in \mathcal{N}(v_{k_1})} \left( \frac{1}{d(v_{k_2})^2} \cdots \sum_{k_l \in \mathcal{N}(v_{k_{l-1}})} \frac{1}{d(v_{k_l})} \right) \right].$$

In the large-dimension limit ( $d_l \rightarrow \infty$ ), each row of  $X^{(l)} W^{(l)}$  and  $\tilde{A} X^{(l)} W^{(l)}$  is asymptotically Gaussian with zero mean and variance proportional to  $\phi(v)$ . Consequently, the expected row-wise quantization parameters for  $X^{(l)}$  and  $X_c^{(l)}$  vary as uniformly continuous functions of  $\phi(\cdot)$ . In particular:

- If  $\phi(u) = \phi(v)$ , then  $\mathbb{E}[s_u] = \mathbb{E}[s_v]$  and  $\mathbb{E}[z_u] = \mathbb{E}[z_v]$ , where  $(s_u, z_u)$  and  $(s_v, z_v)$  are the respective scale and zero-point parameters of nodes  $u$  and  $v$ .
- More generally, if  $|\phi(u) - \phi(v)| < \delta$ , then  $|\mathbb{E}[s_u] - \mathbb{E}[s_v]| < \epsilon$  and  $|\mathbb{E}[z_u] - \mathbb{E}[z_v]| < \epsilon$ , for any desired  $\epsilon > 0$ , by uniform continuity.

Hence,  $\phi$  faithfully reflects the expected row-wise quantization statistics. If two nodes have similar  $\phi$ -values, their quantization scales and zero-points are also similar in expectation. The proof can be found in Appendix B.

**Practical Formulation.** Although one could directly implement the nested sums from Theorem 5.1, this can be cumbersome or expensive, especially for GNNs with multiple layers. Moreover, we want an index applicable across multiple layers and architectures. Our empirical observations suggest that the pairwise first-layer terms capture much of the benefit with minimal computational overhead. Hence, we propose the following lightweight *Topology-Aware Pairwise Index* (TopPIN):

$$\text{TopPIN}(v) = \left( d(v), \frac{1}{d(v)} \sum_{v_k \in \mathcal{N}(v)} \frac{1}{d(v_k)} \right). \quad (13)$$

This design effectively strikes a balance between accuracy and efficiency in inductive GNN quantization tasks. Note that  $\text{TopPIN}(v)$  does not depend on the definition of  $\tilde{A}$ , and thus can be used for various GNNs.

### 5.3. Overall Process

Figure 4 illustrates the overall process of TopGQ. In the calibration phase (Figure 4a), we first compute  $\text{TopPIN}(v)$  for each node  $v$ . Based on these index values, we then calculate node-wise quantization parameters  $(s_v, z_v)$  as described in Section 2.2. If multiple nodes share the same  $\text{TopPIN}$  value, we aggregate their statistics by taking the maximum of their maximum values and the minimum of their minimum values, ensuring the quantization parameters cover the full dynamic range.

During inference, we apply the selective scale absorption method from Section 5.1. When encountering previously unseen nodes under node-wise quantization, we compute  $\text{TopPIN}(v)$  and identify the  $k$ -nearest  $\text{TopPIN}$  groups to retrieve their quantization parameters, subsequently interpolating among these parameters. Leveraging the theoretical

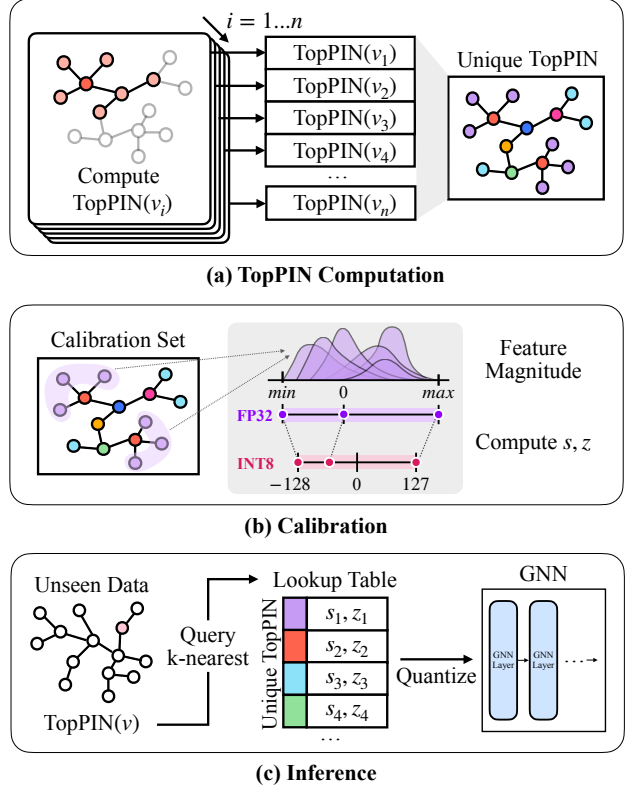


Figure 4. The process of topology-based node grouping. (a) shows group generation using topological characteristics: indegree and Wiener Index. Each color is used to denote each group. (b) shows the calibration process to achieve a set of quantization parameters for each group. (c) demonstrates how inference is done on unseen data by using the quantization parameters of the nearest group.

foundation in Section 5.2, we assume that nodes with similar  $\text{TopPIN}(v)$  values exhibit similar activation distributions and can thus share quantization parameters with minimal accuracy degradation. Moreover, using the  $k$ -nearest groups enhances robustness against minor topological variations, as confirmed by our empirical findings.

## 6. Experiments

We evaluate TopGQ on both node-level and graph-level classification tasks, and compare it with three GNN QAT baselines: Degree-Quant (Tailor et al., 2020), SGQuant (Feng et al., 2020), and  $A^2Q$  (Zhu et al., 2022). For node classification, we use the Cora, CiteSeer, PubMed, PPI, and Reddit datasets; and for graph classification, we use MUTAG and IMDB-BINARY. Unless otherwise noted, we calibrate a fully-trained Floating-point precision GCN (Kipf & Welling, 2016), GIN (Xu et al., 2019), and GraphSAGE (Hamilton et al., 2017) for 4-bit and 8-bit integer quantization; the bitwidth is fixed across all layers for a fair comparison.

Table 1. Comparison of quantization accuracy and time for node classification task.

| Bit  | Method       | Type | Metric          | Cora                    |                         |                         | CiteSeer                 |                         |                          | PubMed                   |                          |                           |
|------|--------------|------|-----------------|-------------------------|-------------------------|-------------------------|--------------------------|-------------------------|--------------------------|--------------------------|--------------------------|---------------------------|
|      |              |      |                 | GCN                     | GIN                     | GraphSAGE               | GCN                      | GIN                     | GraphSAGE                | GCN                      | GIN                      | GraphSAGE                 |
| FP32 | -            | -    | Acc.            | 79.70                   | 79.20                   | 76.36                   | 76.50                    | 76.80                   | 75.82                    | 86.70                    | 88.78                    | 89.08                     |
| INT8 | Degree-Quant | QAT  | Acc.<br>Q. Time | 78.94<br>(11.53s)       | 67.56<br>(18.53s)       | 75.50<br>(23.03s)       | 75.62<br>(25.69s)        | 65.64<br>(50.57s)       | 74.77<br>(69.87s)        | <b>88.26</b><br>(50.63s) | 88.32<br>(188.95s)       | 88.62<br>(181.08s)        |
|      | $A^2Q$       | QAT  | Acc.<br>Q. Time | 79.66<br>(3.53s)        | 69.16<br>(3.29s)        | 76.94<br>(4.46s)        | 75.72<br>(3.46s)         | 67.09<br>(3.97s)        | 75.08<br>(5.27s)         | 85.20<br>(7.99s)         | 88.36<br>(13.06s)        | 88.76<br>(18.28s)         |
|      | SGQuant      | QAT  | Acc.<br>Q. Time | 79.93<br>(6.44s)        | 78.35<br>(5.36s)        | 76.28<br>(8.10s)        | 76.21<br>(10.45s)        | 76.04<br>(11.35s)       | <b>76.06</b><br>(18.19s) | 86.37<br>(14.77s)        | <b>88.89</b><br>(22.63s) | 88.87<br>(29.55s)         |
|      | TopGQ (Ours) | PTQ  | Acc.<br>Q. Time | <b>80.54</b><br>(0.16s) | <b>79.68</b><br>(0.18s) | <b>77.10</b><br>(0.19s) | <b>76.36</b><br>(0.17s)  | <b>76.76</b><br>(0.22s) | 75.78<br>(0.25s)         | 86.66<br>(0.16s)         | 88.70<br>(0.26s)         | <b>89.00</b><br>(0.25s)   |
|      | Degree-Quant | QAT  | Acc.<br>Q. Time | 78.54<br>(11.54s)       | 65.98<br>(18.50s)       | 74.36<br>(23.19s)       | 23.54<br>(25.47s)        | 46.02<br>(49.54s)       | 74.99<br>(69.39s)        | <b>87.78</b><br>(50.20s) | 86.94<br>(190.58s)       | <b>88.58</b><br>(180.73s) |
| INT4 | $A^2Q$       | QAT  | Acc.<br>Q. Time | 50.00<br>(3.60s)        | 68.76<br>(3.18s)        | 74.66<br>(4.48s)        | 43.52<br>(3.49s)         | 62.50<br>(4.00s)        | 73.00<br>(5.21s)         | 70.08<br>(7.89s)         | 84.74<br>(13.15s)        | 85.32<br>(18.41s)         |
|      | SGQuant      | QAT  | Acc.<br>Q. Time | 78.73<br>(6.43s)        | 76.93<br>(5.40s)        | 75.52<br>(8.20s)        | <b>76.31</b><br>(10.41s) | 75.47<br>(11.32s)       | <b>75.94</b><br>(18.11s) | 85.12<br>(14.78s)        | <b>88.33</b><br>(22.61s) | 86.62<br>(29.52s)         |
|      | TopGQ (Ours) | PTQ  | Acc.<br>Q. Time | <b>78.84</b><br>(0.16s) | <b>78.58</b><br>(0.18s) | <b>76.38</b><br>(0.19s) | 75.46<br>(0.16s)         | <b>76.36</b><br>(0.23s) | 75.28<br>(0.24s)         | 85.62<br>(0.16s)         | 87.52<br>(0.26s)         | 87.72<br>(0.24s)          |
|      | Degree-Quant | QAT  | Acc.<br>Q. Time | 78.54<br>(11.54s)       | 65.98<br>(18.50s)       | 74.36<br>(23.19s)       | 23.54<br>(25.47s)        | 46.02<br>(49.54s)       | 74.99<br>(69.39s)        | <b>87.78</b><br>(50.20s) | 86.94<br>(190.58s)       | <b>88.58</b><br>(180.73s) |

Table 2. Performance on node classification task using large graph datasets.

| Dataset | Bit          | Method       | Type         | GCN            |              | GraphSAGE       |          |
|---------|--------------|--------------|--------------|----------------|--------------|-----------------|----------|
|         |              |              |              | Acc.           | Q. Time      | Acc.            | Q. Time  |
| PPI     | FP32         | -            | -            | 71.00          | -            | 94.18           | -        |
|         | INT8         | Degree-Quant | QAT          | 64.55          | (40.21m)     | 67.17           | (42.55m) |
|         |              | $A^2Q$       | QAT          | 40.46          | (4.12m)      | 45.78           | (5.52m)  |
|         |              | SGQuant      | QAT          | <b>73.26</b>   | (5.58m)      | <b>91.33</b>    | (7.76m)  |
|         | TopGQ (Ours) | PTQ          | 70.33        | <b>(1.88s)</b> | 89.76        | <b>(2.36s)</b>  |          |
| INT4    | INT8         | Degree-Quant | QAT          | 48.32          | (45.94m)     | 64.24           | (42.54m) |
|         |              | $A^2Q$       | QAT          | 40.83          | (3.62m)      | 43.12           | (5.53m)  |
|         |              | SGQuant      | QAT          | 55.71          | (5.08m)      | 59.46           | (7.50m)  |
|         | TopGQ (Ours) | PTQ          | <b>58.26</b> | <b>(1.88s)</b> | <b>70.43</b> | <b>(2.41s)</b>  |          |
|         | FP32         | -            | -            | 92.12          | -            | 95.10           | -        |
| Reddit  | INT8         | Degree-Quant | QAT          | 87.01          | (10.59m)     | 90.53           | (16.35h) |
|         |              | $A^2Q$       | QAT          | 73.71          | (4.12m)      | 75.13           | (2.83h)  |
|         |              | SGQuant      | QAT          | <b>92.10</b>   | (4.64m)      | 92.01           | (3.27h)  |
|         | TopGQ (Ours) | PTQ          | 89.49        | <b>(1.88s)</b> | <b>94.62</b> | <b>(35.79s)</b> |          |
|         | INT4         | Degree-Quant | QAT          | 64.18          | (10.55m)     | 89.61           | (16.33h) |
|         |              | $A^2Q$       | QAT          | 23.24          | (4.12m)      | 67.94           | (2.83h)  |
|         |              | SGQuant      | QAT          | 43.00          | (4.84m)      | 87.42           | (3.27h)  |
|         | TopGQ (Ours) | PTQ          | <b>88.53</b> | <b>(1.87s)</b> | <b>90.42</b> | <b>(35.28s)</b> |          |

Further experimental details are provided in the Appendix.

## 6.1. Evaluation Results

Table 1 summarizes the results on the conventional node-level datasets commonly used in prior work, while Table 2 presents the results on larger-scale node-level datasets. Across all these settings, TopGQ consistently achieves comparable or superior accuracy compared to the baselines, and does so with substantially reduced quantization time. On the smaller datasets (Table 1), the overall quantization overhead is naturally limited for all methods, but TopGQ still remains

the fastest. For larger datasets (Table 2), the difference becomes more pronounced:

Taking Reddit with 4-bit GraphSAGE as an example, the best-performing baseline is Degree-Quant, with 89.61% accuracy. However, it suffers from almost 16 hours of quantization time. SGQuant and  $A^2Q$  are faster on quantization, but suffer from severe accuracy drops.

In contrast, TopGQ offers the best of both worlds: high accuracy with an order-of-magnitude faster quantization. TopGQ achieves a significantly higher accuracy of 90.42%, with only 35 seconds of quantization time. This is  $1666\times$  faster than Degree-Quant, and more than  $280\times$  faster than the low-performing baselines (SGQuant and  $A^2Q$ ).

Table 3 presents the graph-level classification results on MUTAG and IMDB-BINARY. Here, we observe a similar trend: The proposed method, TopGQ, significantly improves quantization speed while maintaining competitive classification performance. For instance, Degree-Quant takes 1307 seconds to quantize the GraphSAGE model on IMDB-BINARY, with a significant drop in accuracy of 4.48%p whereas TopGQ quantizes in less than 2 seconds with minimal performance loss ( $<1\%$ p).

While TopGQ taking the least to quantize and can save up to orders of magnitude of time, in many cases TopGQ also shows the best accuracy with minimal degradation compared to FP32. We attribute this to TopGQ’s explicit integration of GNN-specific considerations – leveraging TopPIN to effectively capture local topological structures, while existing QAT baselines largely neglect these properties.

Overall, the experimental results for node and graph classification tasks demonstrate that TopGQ provides a robust

Table 3. Comparison of quantization accuracy and time for graph classification task.

| Bit  | Method       | Type | Metric         | MUTAG                          |                                |                                | IMDB-BINARY                    |                                |                                |
|------|--------------|------|----------------|--------------------------------|--------------------------------|--------------------------------|--------------------------------|--------------------------------|--------------------------------|
|      |              |      |                | GCN                            | GIN                            | GraphSAGE                      | GCN                            | GIN                            | GraphSAGE                      |
| FP32 | -            | -    | Acc.           | 86.94                          | 93.01                          | 86.84                          | 79.28                          | 79.76                          | 77.52                          |
|      | Degree-Quant | QAT  | Acc.<br>Q.Time | 80.91<br>(144.26s)             | 89.53<br>(118.25s)             | 80.24<br>(128.70s)             | 77.32<br>(1610.24s)            | 76.00<br>(1258.45s)            | 73.04<br>(1307.76s)            |
| INT8 | $A^2Q$       | QAT  | Acc.<br>Q.Time | 79.30<br>(49.92s)              | 86.58<br>(49.92s)              | 83.12<br>(50.54s)              | 75.12<br>(221.00s)             | 75.97<br>(250.11s)             | 74.49<br>(266.23s)             |
|      | SGQuant      | QAT  | Acc.<br>Q.Time | 81.87<br>(107.83s)             | 86.50<br>(102.93s)             | 83.53<br>(101.90s)             | 68.28<br>(450.36s)             | 68.26<br>(423.23s)             | 66.36<br>(408.17s)             |
|      | TopGQ (Ours) | PTQ  | Acc.<br>Q.Time | <b>85.33</b><br><b>(1.43s)</b> | <b>92.06</b><br><b>(1.22s)</b> | <b>86.31</b><br><b>(1.26s)</b> | <b>78.82</b><br><b>(2.53s)</b> | <b>79.02</b><br><b>(2.05s)</b> | <b>76.84</b><br><b>(1.93s)</b> |
| INT4 | Degree-Quant | QAT  | Acc.<br>Q.Time | <b>85.27</b><br>(145.56s)      | <b>90.58</b><br>(120.49s)      | 80.66<br>(127.96s)             | <b>76.02</b><br>(1658.08s)     | <b>75.98</b><br>(1269.09s)     | 73.52<br>(1307.76s)            |
|      | $A^2Q$       | QAT  | Acc.<br>Q.Time | 80.99<br>(50.95s)              | 88.78<br>(49.09s)              | 77.99<br>(50.02s)              | 74.09<br>(216.68s)             | 75.62<br>(232.20s)             | 73.92<br>(253.02s)             |
|      | SGQuant      | QAT  | Acc.<br>Q.Time | 81.57<br>(105.25s)             | 87.23<br>(101.82s)             | <b>86.02</b><br>(101.16s)      | 67.64<br>(421.92s)             | 63.72<br>(421.57s)             | 65.56<br>(417.21s)             |
|      | TopGQ (Ours) | PTQ  | Acc.<br>Q.Time | 68.15<br><b>(1.49s)</b>        | 88.03<br><b>(1.23s)</b>        | 83.37<br><b>(1.23s)</b>        | 74.74<br><b>(2.62s)</b>        | 74.08<br><b>(2.13s)</b>        | <b>75.54</b><br><b>(2.00s)</b> |

Table 4. Inference time comparison using GCN.

| Method         | Type | INT8      |         | INT4      |         |
|----------------|------|-----------|---------|-----------|---------|
|                |      | Time (ms) | Speedup | Time (ms) | Speedup |
| FP32           | -    | 23.37     | -       | 23.37     | -       |
| Degree-Quant   | QAT  | 16.05     | 1.46×   | 15.83     | 1.48×   |
| $A^2Q$         | QAT  | 24.18     | 0.97×   | 24.36     | 0.96×   |
| SGQuant        | QAT  | 16.43     | 1.42×   | 16.27     | 1.44×   |
| On-the-fly PTQ | PTQ  | 21.65     | 1.08×   | 21.06     | 1.11×   |
| TopGQ          | PTQ  | 16.07     | 1.45×   | 16.11     | 1.45×   |

Table 5. Comparison of accuracy and node index computation time for various indexing strategies on IMDB-BINARY.

| Bit  | Node Index  | GCN          | GIN          | GraphSAGE    | Time         |
|------|-------------|--------------|--------------|--------------|--------------|
| FP32 | -           | 79.28        | 79.76        | 77.52        | -            |
| INT4 | Naive PTQ   | 60.14        | 56.50        | 74.76        | 0.00s        |
|      | Betweenness | 50.00        | 50.00        | 50.00        | 1.85s        |
|      | Closeness   | 72.90        | 67.78        | 75.36        | 1.48s        |
|      | Katz        | 69.34        | 72.58        | 74.26        | 20.04s       |
|      | TopPIN      | <b>74.74</b> | <b>74.08</b> | <b>75.54</b> | <b>0.16s</b> |

balance between accuracy and quantization speed, making it well-suited for both small and large-scale GNN tasks.

## 6.2. Inference Latency

Table 4 reports the inference latency of TopGQ and competing baselines. A key observation is that  $A^2Q$  and on-the-fly quantization are prohibitively expensive because both methods require scanning each row of the feature matrix to derive quantization parameters—a process that becomes impractical for large-scale graphs (Section 4). These findings highlight the importance of storing quantization parameters and retrieving them through a suitable node index. Notably, TopGQ leverages the lightweight TopPIN mecha-

nism, where index computation incurs negligible overhead, thereby enabling efficient inference.

## 6.3. Analysis on TopPIN

Finally, we assess the effectiveness of TopPIN by comparing it against a naive PTQ strategy as well as commonly used centrality measures, including betweenness, closeness, and Katz centrality (Table 5). We report both the accuracy and the total computation time required to calculate index values for all nodes in the given graph. The naive PTQ approach, lacking any node-wise indexing, suffers from substantial accuracy degradation due to the large variance in per-node magnitudes, as it relies on a single global quantization parameter. Meanwhile, the conventional centrality measures require a global graph traversal for each node index computation. In contrast, TopPIN only depends on 1-hop neighborhood information, thereby significantly reducing the computational overhead. Despite its lightweight design, TopPIN consistently outperforms the more complex centrality baselines, highlighting its practicality and effectiveness as an indexing strategy for GNN quantization.

## 7. Conclusion

In this paper, we study post-training quantization of GNNs. Our method, named TopGQ, leverages topological similarity of the nodes to group activations with similar range, reducing the quantization error. On top of that, we also propose scale absorption, which enables loss-less row-wise quantization of feature matrix for better efficiency. Evaluation reveals that TopGQ reduces quantization time in orders of magnitude scale whilst exceeding QAT performance, achieving best of both worlds.

## Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

## References

- Arazzi, M., Cotogni, M., Nocera, A., and Virgili, L. Predicting tweet engagement with graph neural networks. In *Proceedings of the 2023 ACM International Conference on Multimedia Retrieval*, pp. 172–180, 2023.
- Bai, L., Yao, L., Li, C., Wang, X., and Wang, C. Adaptive graph convolutional recurrent network for traffic forecasting. *Advances in neural information processing systems*, 33:17804–17815, 2020.
- Borgwardt, K. M., Ong, C. S., Schönauer, S., Vishwanathan, S., Smola, A. J., and Krieger, H.-P. Protein function prediction via graph kernels. *Bioinformatics*, 21(suppl.1): i47–i56, 2005.
- Brasoveanu, A. D., Jögl, F., Welke, P., and Thiessen, M. Extending graph neural networks with global features. In *The Second Learning on Graphs Conference*, 2023.
- Cao, D., Wang, Y., Duan, J., Zhang, C., Zhu, X., Huang, C., Tong, Y., Xu, B., Bai, J., Tong, J., et al. Spectral temporal graph neural network for multivariate time-series forecasting. *Advances in neural information processing systems*, 33:17766–17778, 2020.
- Choi, K., Hong, D., Park, N., Kim, Y., and Lee, J. Qimera: Data-free quantization with synthetic boundary supporting samples. In *Advances in Neural Information Processing Systems*, 2021.
- Choukroun, Y., Kravchik, E., Yang, F., and Kisilev, P. Low-bit quantization of neural networks for efficient inference. In *IEEE/CVF International Conference on Computer Vision Workshop*, 2019.
- Eliasof, M., Bodner, B. J., and Treister, E. Haar wavelet feature compression for quantized graph convolutional networks. *IEEE Transactions on Neural Networks and Learning Systems*, 2023.
- Fan, W., Ma, Y., Li, Q., He, Y., Zhao, E., Tang, J., and Yin, D. Graph neural networks for social recommendation. In *The world wide web conference*, pp. 417–426, 2019.
- Feng, B., Wang, Y., Li, X., Yang, S., Peng, X., and Ding, Y. Sgquant: Squeezing the last bit on graph neural networks with specialized quantization. In *2020 IEEE 32nd international conference on tools with artificial intelligence (ICTAI)*, pp. 1044–1052. IEEE, 2020.
- Gao, C., Zheng, Y., Li, N., Li, Y., Qin, Y., Piao, J., Quan, Y., Chang, J., Jin, D., He, X., et al. A survey of graph neural networks for recommender systems: Challenges, methods, and directions. *ACM Transactions on Recommender Systems*, 1(1):1–51, 2023.
- Hamilton, W., Ying, Z., and Leskovec, J. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 2017.
- Hu, M., Sun, D., You, F., and Xiao, H. Hybrid structure encoding graph neural networks with attention mechanism for link prediction. In *2022 IEEE 34th International Conference on Tools with Artificial Intelligence (ICTAI)*, pp. 417–424. IEEE, 2022.
- Huang, L., Zhang, Z., Du, Z., Li, S., Zheng, H., Xie, Y., and Tan, N. Epquant: A graph neural network compression approach based on product quantization. *Neurocomputing*, 503:49–61, 2022.
- Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *CVPR*, 2018.
- Ji, H. G. Graph u-nets. In *Proceedings of the 36th International Conference on Machine Learning, in Proceedings of Machine Learning Research*, 2019.
- Jiang, W. and Luo, J. Graph neural network for traffic forecasting: A survey. *Expert systems with applications*, 207:117921, 2022.
- Kamath, G. Bounds on the expectation of the maximum of samples from a gaussian. URL [http://www.gautamkamath.com/writings/gaussian\\_max.pdf](http://www.gautamkamath.com/writings/gaussian_max.pdf), 10(20-30): 31, 2015.
- Kipf, T. N. and Welling, M. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2016.
- Liu, Z., Chen, C., Yang, X., Zhou, J., Li, X., and Song, L. Heterogeneous graph neural networks for malicious account detection. In *Proceedings of the 27th ACM international conference on information and knowledge management*, pp. 2077–2085, 2018.
- Lu, M., Han, Z., Rao, S. X., Zhang, Z., Zhao, Y., Shan, Y., Raghunathan, R., Zhang, C., and Jiang, J. Bright-graph neural networks in real-time fraud detection. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, pp. 3342–3351, 2022.

- Pal, A., Eksombatchai, C., Zhou, Y., Zhao, B., Rosenberg, C., and Leskovec, J. Pinnorsage: Multi-modal user embedding framework for recommendations at pinterest. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2311–2320, 2020.
- Qiu, J., Tang, J., Ma, H., Dong, Y., Wang, K., and Tang, J. Deepinf: Social influence prediction with deep learning. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 2110–2119, 2018.
- Sharma, A., Sharma, A., Nikashina, P., Gavrilenko, V., Tselikh, A., Bozhenyuk, A., Masud, M., and Meshref, H. A graph neural network (gnn)-based approach for real-time estimation of traffic speed in sustainable smart cities. *Sustainability*, 15(15):11893, 2023.
- Tailor, S. A., Fernandez-Marques, J., and Lane, N. D. Degree-quant: Quantization-aware training for graph neural networks. In *International Conference on Learning Representations*, 2020.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. Graph attention networks. In *International Conference on Learning Representations*, 2018.
- Wale, N., Watson, I. A., and Karypis, G. Comparison of descriptor spaces for chemical compound retrieval and classification. *Knowledge and Information Systems*, 14: 347–375, 2008.
- Wang, S., Eravci, B., Guliyev, R., and Ferhatosmanoglu, H. Low-bit quantization for deep graph neural networks with smoothness-aware message propagation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, pp. 2626–2636, 2023.
- Wei, X., Zhang, Y., Zhang, X., Gong, R., Zhang, S., Zhang, Q., Yu, F., and Liu, X. Outlier suppression: Pushing the limit of low-bit transformer language models. *Advances in Neural Information Processing Systems*, 35:17402–17414, 2022.
- Wu, Z., Ramsundar, B., Feinberg, E. N., Gomes, J., Geniesse, C., Pappu, A. S., Leswing, K., and Pande, V. Moleculenet: a benchmark for molecular machine learning. *Chemical science*, 9(2):513–530, 2018.
- Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pp. 38087–38099. PMLR, 2023.
- Xu, K., Hu, W., Leskovec, J., and Jegelka, S. How powerful are graph neural networks? In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. URL <https://openreview.net/forum?id=ryGs6iA5Km>.
- Xu, Z., Lai, S., Liu, X., Abuadbbba, A., Yuan, X., and Yi, X. {OblivGNN}: Oblivious inference on transductive and inductive graph neural network. In *USENIX Security*, 2024.
- Yao, Y., Kamani, M. M., Cheng, Z., Chen, L., Joe-Wong, C., and Liu, T. Fedrule: Federated rule recommendation system with graph neural networks. In *Proceedings of the 8th ACM/IEEE Conference on Internet of Things Design and Implementation*, pp. 197–208, 2023.
- You, J., Gomes-Selman, J. M., Ying, R., and Leskovec, J. Identity-aware graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pp. 10737–10745, 2021.
- Zhang, L. and Lu, H. A feature-importance-aware and robust aggregator for gcn. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pp. 1813–1822, 2020.
- Zhang, Y., Wu, L., Shen, Q., Pang, Y., Wei, Z., Xu, F., Chang, E., and Long, B. Graph learning augmented heterogeneous graph neural network for social recommendation. *ACM Transactions on Recommender Systems*, 1(4):1–22, 2023.
- Zhao, R., Hu, Y., Dotzel, J., De Sa, C., and Zhang, Z. Improving neural network quantization without retraining using outlier channel splitting. In *International Conference on Machine Learning*, 2019.
- Zhou, H., Srivastava, A., Zeng, H., Kannan, R., and Prasanna, V. Accelerating large scale real-time gnn inference using channel pruning. *arXiv preprint arXiv:2105.04528*, 2021.
- Zhu, Z., Li, F., Mo, Z., Hu, Q., Li, G., Liu, Z., Liang, X., and Cheng, J. A<sup>2</sup>q: Aggregation-aware quantization for graph neural networks. In *The Eleventh International Conference on Learning Representations*, 2022.

## A. Additional Experimental Settings

We report evaluation results on two representative graph processing tasks: Node-level classification, graph-level classification. For node-level classification, we compare the validation accuracy of Cora, Citeseer, PubMed, PPI, and Reddit, in inductive setting. For graph-level classification, we choose MUTAG and IMDB-BINARY datasets to evaluate the inductive inference performance of quantized GNNs.

We compare TopGQ with three graph quantization baselines using QAT approaches: Degree-Quant (Tailor et al., 2020), SGQuant (Feng et al., 2020), and  $A^2Q$  (Zhu et al., 2022). To ensure a fair comparison, a fixed-precision quantization for both SGQuant and  $A^2Q$  when attaining experiment results. We report quantized accuracy of GCN (Kipf & Welling, 2016), GIN (Xu et al., 2019), and GraphSAGE (Hamilton et al., 2017) architectures with 4-bit and 8-bit integer quantization. For a fair comparison, we apply the same bitwidth for all layers, including aggregation and combination.

All experiments are conducted on a server with a single A6000 GPU, RTX 4090 GPU, and Intel(R) Xeon(R) Gold 6442Y CPU. We implement our algorithm on PyG library v2.6.0 with PyTorch v2.2.1. In the index computation time comparison, we use the SciPy library.

## B. Proof of Theorem 5.1

**Notations.** We consider the  $L$ -layer GNN

$$X^{(l+1)} = \sigma(\tilde{A} X^{(l)} W^{(l)}), \quad l = 0, \dots, L-1,$$

where  $\sigma(\cdot)$  is the ReLU activation, and

$$\tilde{A} \in \left\{ A + I_n, \quad D^{-\frac{1}{2}}(A + I_n)D^{-\frac{1}{2}} \right\}.$$

Assume that each entry of the initial feature matrix  $X^{(0)} \in \mathbb{R}^{n \times d_0}$  is drawn i.i.d. from a zero-mean distribution  $\mathcal{D}_x(0, \sigma_x^2)$ , and each entry of the weight matrices  $W^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$  is drawn i.i.d. from a zero-mean distribution  $\mathcal{D}_l(0, \sigma_l^2)$ . Denote

$$Z^{(l)} = X^{(l)} W^{(l)}, \quad Y^{(l)} = \tilde{A} Z^{(l)}, \quad X^{(l+1)} = \sigma(Y^{(l)}).$$

for notational simplicity. We prove in several steps that each row of  $Z^{(l)}$  and  $Y^{(l)}$  is (approximately) a zero-mean Gaussian whose variance depends only on a node-specific function  $\phi(\cdot)$ , and that this leads to the stated continuity property of expected quantization parameters.

**Step 1: Propagation for 1st layer of GNN** Consider  $Z^{(0)} = X^{(0)} W^{(0)}$ . For each entry,

$$Z_{ij}^{(0)} = \sum_{\alpha=1}^{d_0} X_{i\alpha}^{(0)} W_{\alpha j}^{(0)}.$$

Since  $X_{i\alpha}^{(0)}$  and  $W_{\alpha j}^{(0)}$  are i.i.d. with zero mean and finite variance, it follows that

$$\mathbb{E}[X_{i\alpha}^{(0)} W_{\alpha j}^{(0)}] = 0, \quad \text{Var}(X_{i\alpha}^{(0)} W_{\alpha j}^{(0)}) = \mathbb{E}[(X_{i\alpha}^{(0)})^2] \mathbb{E}[(W_{\alpha j}^{(0)})^2] = \sigma_x^2 \sigma_0^2$$

By the Central Limit Theorem (CLT),  $Z_{ij}^{(0)}$  is approximately  $\mathcal{N}(0, d_0 \sigma_x^2 \sigma_0^2)$ .

Next, we consider the multiplication by the augmented adjacency matrix. For instance,  $\tilde{A} = D^{-\frac{1}{2}}(I + A)D^{-\frac{1}{2}}$ .

$$Y^{(0)} = \tilde{A} Z^{(0)}, \quad Y_{ij}^{(0)} = \sum_{\alpha=1}^n \tilde{A}_{i\alpha} Z_{\alpha j}^{(0)} = \sum_{\{k|v_k \in \mathcal{N}(v_i)\}} \frac{1}{\sqrt{d(v_i) d(v_k)}} Z_{kj}^{(0)}.$$

Summing independent Gaussians preserves Gaussianity and yields

$$Y_{ij}^{(0)} \sim \mathcal{N}(0, [d_0 \sigma_x^2 \sigma_0^2] \frac{1}{d(v_i)} \sum_{v_k \in \mathcal{N}(v_i)} \frac{1}{d(v_k)}).$$

**Step 2: Subsequent layers and ReLU.** Next we consider for the second layer of GNN. Focusing on a particular entry  $Z_{ij}^{(1)}$ , we can write

$$Z_{ij}^{(1)} = \sum_{\alpha=1}^{d_1} X_{i\alpha}^{(1)} W_{\alpha j}^{(1)} = \sum_{\alpha=1}^{d_1} \sigma(Y_{i\alpha}^{(1)}) W_{\alpha j}^{(1)}.$$

Since  $W_{\alpha j}^{(1)} \sim \mathcal{N}(0, \sigma_1^2)$ , we have :

$$\mathbb{E}[X_{i\alpha}^{(1)} W_{\alpha j}^{(1)}] = 0, \quad \text{Var}(X_{i\alpha}^{(1)} W_{\alpha j}^{(1)}) = \mathbb{E}[(\sigma(Y_{i\alpha}^{(1)}))^2] \sigma_1^2 = \frac{[d_0 \sigma_x^2 \sigma_0^2] \frac{1}{d(v_i)} \sum_{v_k \in \mathcal{N}(v_i)} \frac{1}{d(v_k)}}{2} \sigma_1^2.$$

Summing over  $\alpha = 1, \dots, d_1$  and again invoking the CLT, we arrive at

$$Z_{ij}^{(1)} \sim \mathcal{N}(0, [\frac{d_0 d_1 \sigma_x^2 \sigma_0^2 \sigma_1^2}{2}] \frac{1}{d(v_i)} \sum_{v_k \in \mathcal{N}(v_i)} \frac{1}{d(v_k)}).$$

We then multiply by the adjacency matrix  $\tilde{A}$  as before,

$$Y^{(1)} = \tilde{A} Z^{(1)}, \quad Y_{ij}^{(1)} = \sum_{\alpha=1}^n \tilde{A}_{i\alpha} Z_{\alpha j}^{(1)} = \sum_{\{k_1 | v_{k_1} \in \mathcal{N}(v_i)\}} \frac{1}{\sqrt{d(v_i) d(v_{k_1})}} Z_{k_1 j}^{(1)}.$$

Thus for  $Y_{ij}^{(1)}$  we have

$$Y_{ij}^{(1)} \sim \mathcal{N}(0, [\frac{d_0 d_1 \sigma_x^2 \sigma_0^2 \sigma_1^2}{2}] \frac{1}{d(v_i)} \sum_{v_{k_1} \in \mathcal{N}(v_i)} \frac{1}{d(v_{k_1})^2} \sum_{v_{k_2} \in \mathcal{N}(v_{k_1})} \frac{1}{d(v_{k_2})}).$$

One can easily see that through repetition we have

$$\begin{aligned} \phi(v_i) &= \frac{1}{d(v_i)} \sum_{k_1 \in \mathcal{N}(v_i)} \left[ \frac{1}{d(v_{k_1})^2} \sum_{k_2 \in \mathcal{N}(v_{k_1})} \left( \frac{1}{d(v_{k_2})^2} \cdots \sum_{k_l \in \mathcal{N}(v_{k_{l-1}})} \frac{1}{d(v_{k_l})} \right) \right] \\ Y_{ij}^{(l-1)} &\sim \mathcal{N} \left( 0, \frac{(\prod_{m=0}^{l-1} d_m) \sigma_x^2 (\prod_{m=0}^{l-1} \sigma_m^2)}{2^{l-1}} \cdot \phi(v_i) \right) \\ Z_{ij}^{(l)} &\sim \mathcal{N} \left( 0, \frac{(\prod_{m=0}^l d_m) \sigma_x^2 (\prod_{m=0}^l \sigma_m^2)}{2^l} \cdot \phi(v_i) \right) \end{aligned}$$

that is the distribution for each row of  $Z^{(l)}$  and  $Y^{(l)}$  are zero-mean Gaussian whose variance depends on the corresponding node-specific scalar function which is a constant factor of  $\phi$ .

**Step 3 : unnormalized adjacency** For the case  $\tilde{A} = A + I_n$ , since each neighbor contribution is no longer scaled, the entire degree-based weighting disappears. That is,

$$\phi(v_i) = \sum_{v_{k_1} \in \mathcal{N}(v_i)} \sum_{v_{k_2} \in \mathcal{N}(v_{k_1})} \cdots \sum_{v_{k_l} \in \mathcal{N}(v_{k_{l-1}})} 1.$$

<sup>1</sup>For a ReLU applied zero mean gaussian random variable  $Y \sim \mathcal{N}(0, \sigma_y^2)$ ,

$$\mathbb{E}[(\text{ReLU}(Y))^2] = \int_0^\infty y^2 \frac{1}{\sigma_y \sqrt{2\pi}} e^{-y^2/(2\sigma_y^2)} dy = \frac{\sigma_y^2}{2}$$

$$Y_{ij}^{(l-1)} \sim \mathcal{N}\left(0, \frac{\left(\prod_{m=0}^{l-1} d_m\right) \sigma_x^2 \left(\prod_{m=0}^{l-1} \sigma_m^2\right)}{2^{l-1}} \cdot \phi(v_i)\right)$$

$$Z_{ij}^{(l)} \sim \mathcal{N}\left(0, \frac{\left(\prod_{m=0}^l d_m\right) \sigma_x^2 \left(\prod_{m=0}^l \sigma_m^2\right)}{2^l} \cdot \phi(v_i)\right)$$

holds.

**Step 4: Relation to quantization parameters.** Let  $M \in \mathbb{R}^{n \times d}$  be any matrix whose  $i$ th row is comprised of i.i.d. random variables with distribution  $\mathcal{N}(0, \sigma(i)^2)$ . Define the row-wise quantization parameters  $(s_i, z_i)$  for this row by

$$s_i = \frac{\max_{1 \leq j \leq d} M_{ij} - \min_{1 \leq j \leq d} M_{ij}}{q_{\max} - q_{\min}} \quad \text{and} \quad z_i = \min_{1 \leq j \leq d} M_{ij} - s_i q_{\min},$$

where  $q_{\min}$  and  $q_{\max}$  are fixed integers. The expectation of  $\max_j M_{ij}$  and  $\min_j M_{ij}$  can be expressed via the classical order-statistics integrals<sup>2</sup>. One has

$$\mathbb{E}[\max_{1 \leq j \leq d} M_{ij}] = \int_{-\infty}^{\infty} [1 - F_X(x)]^d dx,$$

where  $F_X$  is the cumulative distribution function of  $\mathcal{N}(0, \sigma(i)^2)$ .  $F_X$  is a continuous function of  $\sigma(i)$ . Also, this integral is absolutely convergent, implying that  $\mathbb{E}[\max_j M_{ij}]$  is a continuous function of  $\sigma(i)$ . A similar argument shows that  $\mathbb{E}[\min_j M_{ij}]$  is also continuous in  $\sigma(i)$ . Consequently,

$$\mathbb{E}[s_i] = g(\sigma(i)), \quad \mathbb{E}[z_i] = h(\sigma(i)),$$

for some continuous functions  $g, h$ . From Steps 1–3, we showed that the row variance  $\sigma(i)^2$  in our GNN setting is proportional to a node-dependent scalar  $\phi(v_i)$ . Hence,

$$\mathbb{E}[s_i] = G(\phi(v_i)), \quad \mathbb{E}[z_i] = H(\phi(v_i)),$$

for some continuous functions  $G, H$ . Next, let

$$A = \max_{v \in V} \phi(v).$$

Since  $G, H$  are continuous on a compact set  $[0, A]$ ,  $G$  and  $H$  are uniformly continuous on  $[0, A]$ . Therefore,

$$|\phi(u) - \phi(v)| < \delta \implies |G(\phi(u)) - G(\phi(v))| < \epsilon \quad \text{and} \quad |H(\phi(u)) - H(\phi(v))| < \epsilon,$$

for any desired  $\epsilon > 0$ . In particular, if  $\phi(u) = \phi(v)$ , then  $\mathbb{E}[s_u] = \mathbb{E}[s_v]$  and  $\mathbb{E}[z_u] = \mathbb{E}[z_v]$ .

*Remark.* In the case that  $M_{ij} = \text{ReLU}(Y_{ij})$  for i.i.d. zero mean Gaussian  $Y_{ij}$ , as  $d \rightarrow \infty$ , the probability of at least one entry being zero in each row goes to one, so  $\mathbb{E}[\min_j \text{ReLU}(Y_{ij})] \rightarrow 0$ . Thus the above argument for scale parameters is valid for all matrices  $X^{(l)}$  and  $Z^{(l)} = X_c^{(l)}$ .

This completes the proof.

## C. Quantization Trade-off Analysis

Here, we present a comprehensive analysis regarding the trade-offs of quantization and provide a theoretical analysis of computation cost and storage consumption. The analysis is shown in Table 6.

<sup>2</sup>As  $d \rightarrow \infty$ , it is known that

$$\frac{\mathbb{E}[\max_j M_{ij}]}{\sqrt{\ln d}} \rightarrow \sqrt{2}\sigma$$

In other words, the integral is approximately  $\sigma\sqrt{2\ln d}$  (Kamath, 2015).

Table 6. Comparison of theoretical costs and storage for different methods.

| Metrics          | Theoretical Cost                                  | Theoretical Storage                              |
|------------------|---------------------------------------------------|--------------------------------------------------|
| FP32             | $O_{FP}(N^2F_1 + NF_1F_2)$                        | $O_{FP}(E + F_1F_2 + NF_0)$                      |
| Degree-Quant     | $O_{INT}(N^2F_1 + NF_1F_2) + O_{FP_{elem}}(NF_2)$ | $O_{INT}(E + F_1F_2 + NF_0) + O_{FP}(1)$         |
| Degree-Quant-PTQ | $O_{INT}(N^2F_1 + NF_1F_2) + O_{FP_{elem}}(NF_2)$ | $O_{INT}(E + F_1F_2 + NF_0) + O_{FP}(1)$         |
| TopGQ            | $O_{INT}(N^2F_1 + NF_1F_2) + O_{FP_{elem}}(NF_2)$ | $O_{INT}(E + F_1F_2 + NF_0) + O_{FP}(N_T + F_2)$ |

TopGQ finds a good balance between reducing quantization time and preserving accuracy, while other choices in FP32, Degree-Quant, TopGQ demonstrate disadvantages in either accuracy, time, or memory. FP32 suffers from the expensive costs of computation and storage. While Degree-Quant alleviates this cost via quantization, the long quantization time is required to obtain the benefits. TopGQ is free from the quantization time problem but at the cost of considerable performance degradation. TopGQ aims to find the best way of addressing each issue by leveraging topological node similarities with an additional amount of storage cost.

As for the theoretical costs (Table 6), we assume GNN layer propagation as AXW operation, with  $A \in \mathbb{R}^{N \times N}$ ,  $X \in \mathbb{R}^{N \times F_1}$ ,  $W \in \mathbb{R}^{F_1 \times F_2}$  with initial dataset size of  $N * F_0$ . We note computation and storage costs of FP and Int operation as follows:

- $O_{FP}()$ : Complexity for floating-point operations / Storage complexity for floating-point values.
- $O_{FP_{elem}}()$ : Complexity for element-wise floating-point operations.
- $O_{INT}()$ : Complexity for fixed-point operations / Storage complexity for fixed-point values.

The computation cost shows that quantization converts the expensive floating-point matrix multiplication into integer operations. The additional floating-point cost comes from converting integer outputs back to floating-point values. The theoretical analysis is based on (Zhu et al., 2022).